

Spring 2026 MSML606 HW5 (deadline: 4/19, Sunday)

Overview

This assignment focuses on understanding of NP-completeness and order statistics. You are offered questions and practice problems that can prepare you for the final exam. Your submission includes a report that must answer both Parts I and II.

Instructions:

- Do not directly copy content from textbooks, other individuals' work, AI-generated outputs, online sources, or any material that isn't your own. While collaboration and consulting various resources are encouraged, it's essential that you express your understanding in your own words, reflecting your personal learning and insights.
- Add justifications/explanations to all answers, as there will be points awarded for that. Please write in your own words.

Important!

Before submitting your work, review the external-source policy in the syllabus. **You must either (1) cite any external sources you used, including AI tools, or (2) explicitly state that you did not use any. Submissions without this AI usage statement will receive no points!!!**

Part I (NP-completeness)

Deliverables: Include your written work with explanations in your own words as a part of the report. Alternatively, you can create a separate document named "NP-completeness" for your Part I submission.

Instructions: The questions below explore the relationship between ML and NP-completeness. You may choose any number of questions to answer, earning up to 1 point per problem.

- If a question is difficult to generalize across all ML models, you may focus on specific examples such as classification, object detection, language translation, generative AI, etc.
- **Clearly state any assumptions you make.**
- Answers should be at least one paragraph but no longer than one page per question. Avoid simple yes/no responses. Provide reasoning and explanations.

Important! You are encouraged to use online resources, but **you must include citations and explain concepts in your own words**. AI tools may not be used to generate direct answers. If you use AI to find references, you must include the AI model name, the prompt you used, and the

response received in a separate document. Refer to the syllabus for details on Academic Integrity and course policies.

Question 1. TSP problem. You need to find the shortest route, visiting all cities exactly once and returning to the start. Match the following three approaches to the strongest guarantee they provide:

Algorithm Approach:

- A. Exact TSP Solver
- B. Greedy Algorithm
- C. Pure ML Model

Options:

- (1) Valid tour — visits every city exactly once and returns to start
- (2) Good tour — valid + reasonably short (guarantees proven bound, e.g., no more than 1.5x optimal solution)
- (3) Optimal tour — valid + provably shortest possible
- (4) None of the above

Provide justification.

Question 2. For an NP-complete problem like TSP, brute force is $O(n!)$. A trained ML model can produce a solution in milliseconds. Does that mean ML solved TSP efficiently? What's the difference between a good solution and an optimal solution?

Question 3. If ML can approximate an NP-hard problem well in practice, does that mean the problem is no longer hard? What does NP-hardness actually say about ML approximations?

Question 4. How would you evaluate whether an ML solution to an NP-hard problem is good if you can't efficiently compute the optimal solution to compare against?

Rubric for Part I: (Maximum Score: 4 Marks)

Problem	Criteria	Marks
Questions 1-4	- Thoughtful answer is provided (1 point) - More than a paragraph but less than a page (<u>must have</u>) - AI and external resources are clearly stated (<u>must have</u>) - All assumptions are clearly stated (<u>must have</u>)	1 per question

Part II (Order statistics)

Deliverables: Include your written work with explanations in your own words as a part of the report. Alternatively, you can create a separate document named “Order Statistics” for your Part II submission.

All the concepts needed for the written portion (traces, recurrences, recursion trees) are demonstrated in the lecture slides. Please review those examples before using external tools!

Problem 1. Use QuickSelect algorithm to trace:

- for a random input
- and for a sorted input

Requirements: use at least 6-8 elements so your traces are meaningfully illustrative

Problem 2. Suppose we are using the QuickSelect algorithm on a worst case input sequence. Pivot always lands at the smallest or largest element.

- Write the recurrence relation $T(n)$
- Draw recursion trees
- Solve the recurrence

Problem 3. Suppose we are using the Median-of-medians algorithm.

- Write the recurrence relation $T(n)$
- Draw recursion trees
- Solve the recurrence

Problem 4. Suppose we develop a pivot-selection algorithm (like Median-of-medians) that guarantees the pivot falls within the 10th to 90th percentile. That means that at least 10% of the elements are discarded in every recursive step. Assume that Partitioning still takes $O(n)$.

- Write the recurrence relation $T(n)$ representing the worst-case running time for this algorithm.
- Draw recursion trees
- Solve the recurrence

Problem 5. (Not graded and no submission required, but maybe useful for final prep)

Now, consider a different strategy: we discard a fixed constant k of elements at each time step (e.g., $k=10$, $k=100$, or larger). How does this change the worst-case complexity?

- **NOTE:** Remember that for Big-O, we evaluate behavior as n approaches infinity. Even if k is a very large constant (like 1 million), it eventually becomes negligible compared to a sufficiently large n

- **Base case:** when the remaining elements $n \leq k$, the recursion stops. In practice, sorting 1 million elements takes a noticeable amount of time. However, k is a constant that does not grow when n grows, we treat this work as $O(1)$ in our occurrence.

Rubric for Part II: (Maximum Score: 9.5 Marks)

Problem	Criteria	Marks
Problem 1: QuickSelect tracing	- Shows tracing for sorted input (1 Marks) - Shows tracing for random input (1 Marks)	2
Problem 2: QuickSelect Recurrence Relation	- Shows recurrence relation (0.5 Marks) - Draws a recurrence tree - Recursion tree with at least 3 levels - (0.5 Marks) - Includes the number of nodes at each level (0.5 Marks) - Shows the work done at each level (0.5 Marks) - Solves the recurrence (0.5 Marks)	2.5
Problem 3: Median-of-medians Recurrence Relation	- Shows recurrence relation (0.5 Marks) - Draws a recurrence tree - Recursion tree with at least 3 levels - (0.5 Marks) - Includes the number of nodes at each level (0.5 Marks) - Shows the work done at each level (0.5 Marks) - Solves the recurrence (0.5 Marks)	2.5
Problem 4: Custom Algorithm Recurrence Relation	- Writes worst-case recurrence relation (1 Marks) - Draws a recurrence tree (0.5 Marks) - Recursion tree with at least 3 levels - (0.5 Marks) - Includes the number of nodes at each level (0.5 Marks) - Shows the work done at each level (0.5 Marks) - Solves the recurrence (0.5 Marks)	2.5
Problem 5: Optional	Optional, no submission required	Not Graded

Part III (Optional Questions, Not Graded)

The following problems are not graded and require no submission. However, these concepts are still important and will be helpful for building intuition before the exam (hint-hint).

Problem 1. Let's consider the following data structures: unsorted array, sorted array, hash table, Balanced BST, Min-Heap or max-Heap. For each data structure describe:

- How would you find the i -th smallest element using that structure?
- Time complexity of your approach
- For which values of i (or which scenarios) this structure performs well or poorly
- How it compares to QuickSelect

Problem 2. Solve the following Leetcode problems:

- K^{th} Largest number in Array:
<https://leetcode.com/problems/kth-largest-element-in-an-array/>
- K^{th} Largest Element in a Stream (this is relevant to the discussion we had in class)
<https://leetcode.com/problems/kth-largest-element-in-a-stream/editorial/>
- K^{th} Smallest Element in a BST
<https://leetcode.com/problems/kth-smallest-element-in-a-bst/description/>